AI Assisted Coding – Lab Assignment 6.3

Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals

Name: v.Tharun varma

HT.No: 2303A52265

Batch: 45

Task 1: Student Class (Object-Oriented Programming)

Scenario

You are developing a simple student information management module.

Task

• Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.

• The class should include attributes such as name, roll number, and branch.

• Add a method display_details() to print student information.

• Execute the code and verify the output.

• Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

• A Python class with a constructor (__init__) and a display_details() method.

• Sample object creation and output displayed on the console.

• Brief analysis of AI-generated code.

Task Understanding

The goal was to create a simple class representing a student, including attributes and a method to display details. This tests AI's ability to generate object-oriented structures.

Steps Performed

- Asked AI tool to generate a class with required attributes.
- AI completed constructor and method definitions.
- Program was executed to verify output.

Prompt:

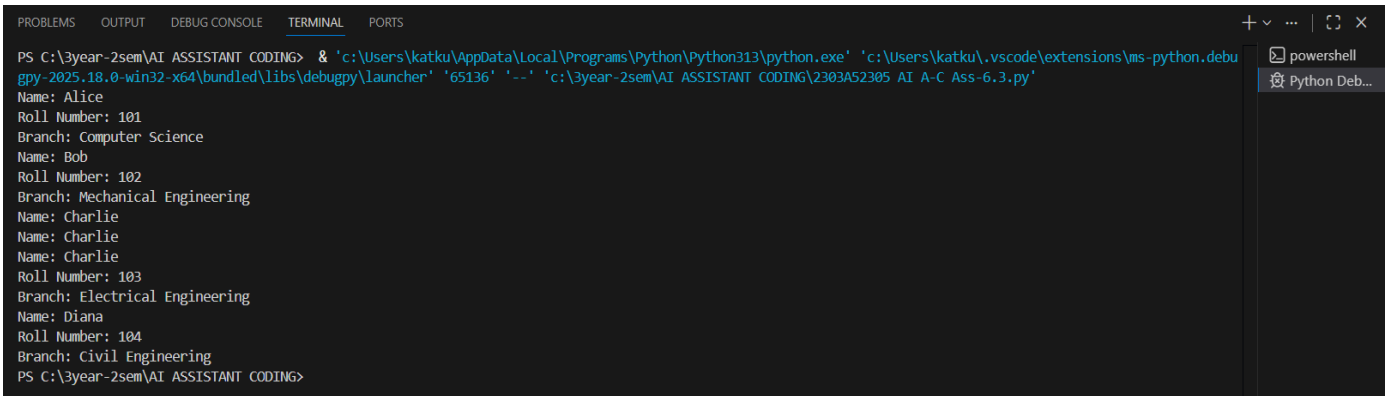
Generate a Python Student class with attributes name, roll number, branch, and a method display_details()

Code

```
class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print(f"Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Branch: {self.branch}")
# Sample object creation
student1 = Student("Alice", "101", "Computer Science")
student1.display_details()
# Sample object creation
student2 = Student("Bob", "102", "Mechanical Engineering")
student2.display_details()
# Sample object creation
student3 = Student("Charlie", "103", "Electrical Engineering")
student3.display_details()
# Sample object creation
student4 = Student("Diana", "104", "Civil Engineering")
student4.display_details()
```

Output



Explanation

AI generated a class structure using Python OOP conventions. The constructor initializes student data, and the method prints stored information. This demonstrates AI's understanding of class syntax and method usage.

Analysis of AI-generated code

- The AI-generated code for the Student class is clear and well-structured.
- It includes an __init__ method to initialize the attributes and a display_details() method to
- print the student information in a readable format.
- The sample object creation demonstrates how to create instances of the Student class and call the display_details() method to display their information.

Observation

- AI successfully structured class components.
- Code readability was good, but validation checks (like input types) were missing.
- Shows AI is strong in syntax but weaker in robustness.

Task 2: Loop-Based Multiples of a Number

Scenario

You are writing a utility function to display multiples of a given number.

Task

• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.

• Analyze the generated loop logic.

• Ask the AI to generate the same functionality using another controlled looping structure (e.g.,while instead of for).

Expected Output #2

• Correct loop-based Python implementation.

• Output showing the first 10 multiples of a number.

• Comparison and analysis of different looping approaches.

Task Understanding

We needed to generate logic to print the first 10 multiples of a number using loops.

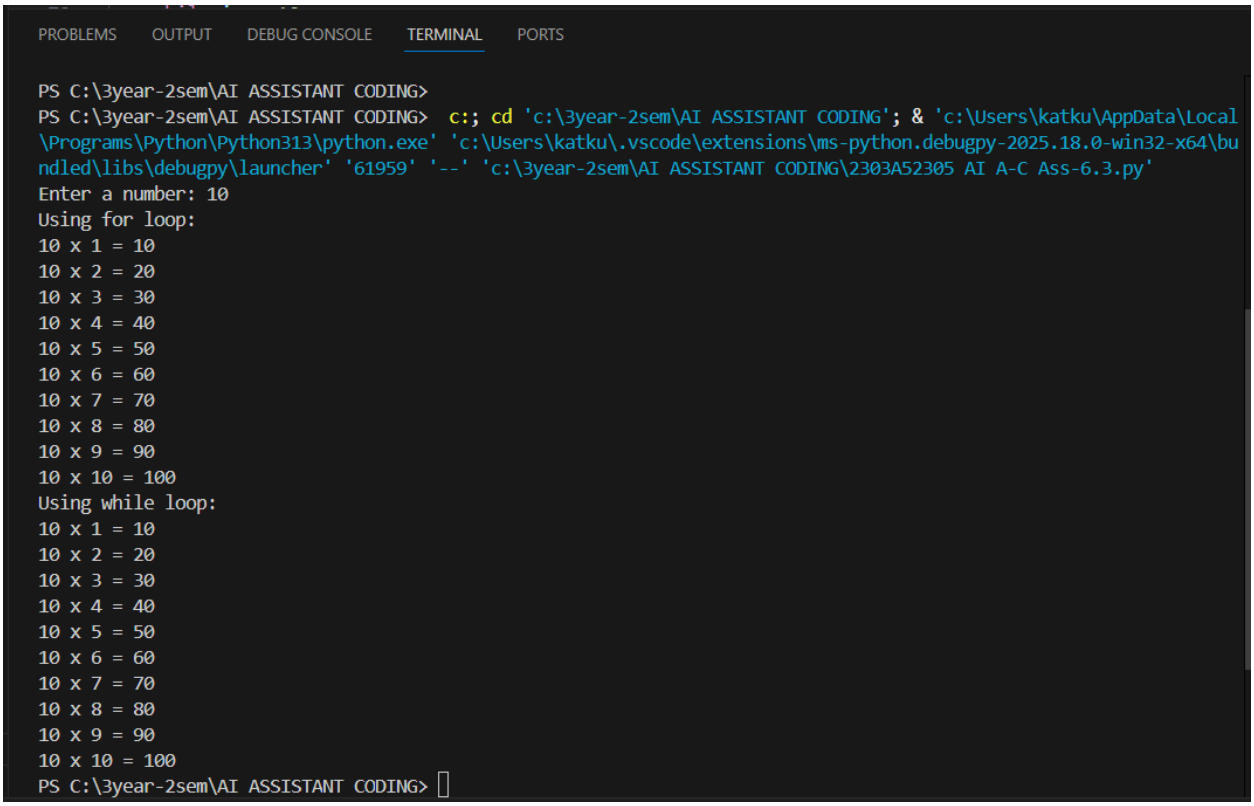
Prompt

Generate a Python function that prints first 10 multiples of a number using a loop.

Code

```
def print_multiples_for_loop(number):
    for i in range(1, 11):
        print(f"{number} x {i} = {number * i}")
def print_multiples_while_loop(number):
    i = 1
    while i <= 10:
        print(f"{number} x {i} = {number * i}")
        i += 1
num = int(input("Enter a number: "))
print("Using for loop:")
print_multiples_for_loop(num)
print("Using while loop:")
print_multiples_while_loop(num)
```

Output



Steps Performed

- AI generated a for-loop solution.
- AI was asked to provide an alternative using a while loop.

Explanation

Both loops performed iterative multiplication. The for-loop offered concise logic, while the while-loop provided more control over iteration.

Analysis of different looping approaches

- Both the for loop and while loop implementations correctly print the first 10 multiples of the given number.
- The for loop is more concise and easier to read for this specific task, as it directly
- iterates over a range of numbers.
- The while loop provides more control over the loop variable but requires manual
- incrementing, which can lead to errors if not handled properly.

Observation

- AI handles basic loop structures effectively.
- It prefers commonly used constructs like for-loops.
- No input validation was added automatically.

Task 3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups(e.g., child, teenager, adult, senior).

• Analyze the generated conditions and logic.

• Ask the AI to generate the same classification using alternative conditional structures (e.g.,simplified conditions or dictionary-based logic).

Expected Output #3

• A Python function that classifies age into appropriate groups.

• Clear and correct conditional logic.

• Explanation of how the conditions work.

Task Understanding

The task required categorizing ages into groups using conditional logic.

Steps Performed

- AI generated nested conditional statements.
- Alternative simplified logic was requested.

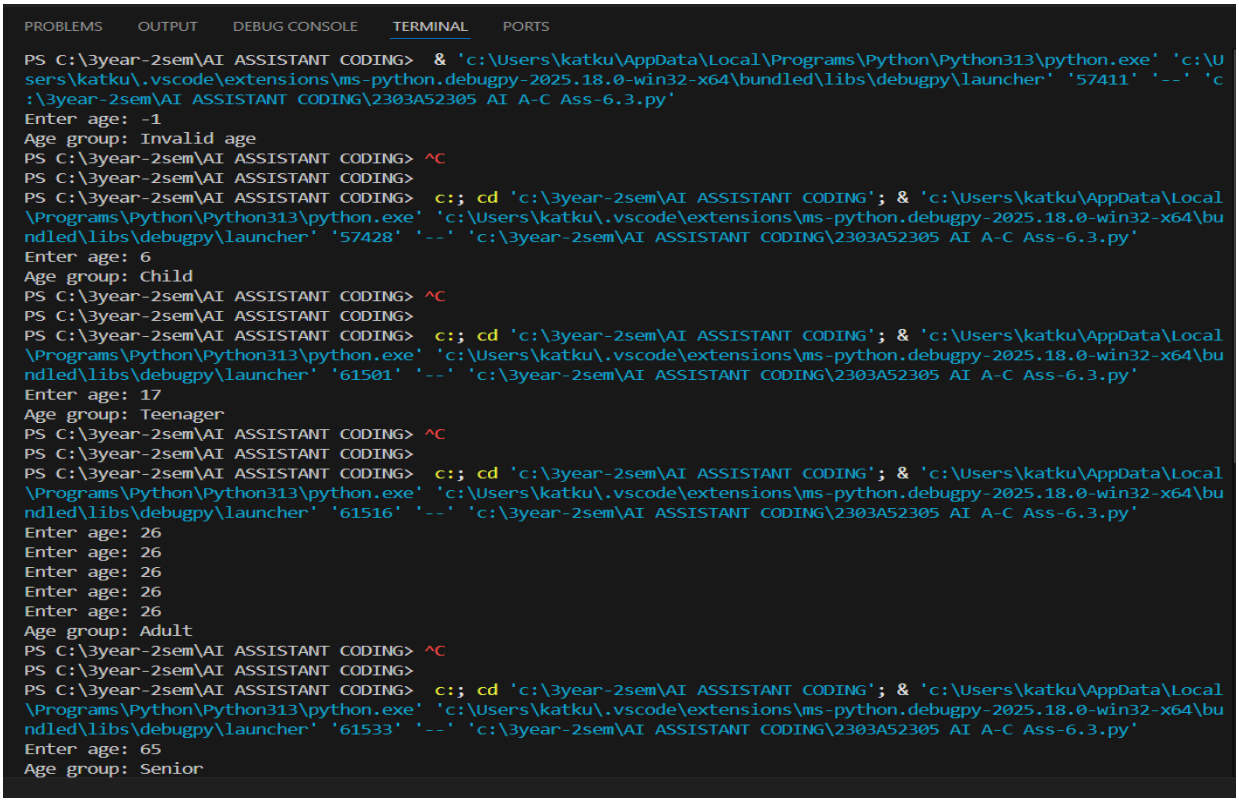
Prompt

Generate code for nested if-elif-else to classify age.

Code:

```
def classify_age(age):
    if age < 0:
        return "Invalid age"
    elif age <= 12:
        return "Child"
    elif age <= 19:
        return "Teenager"
    elif age <= 59:
        return "Adult"
    else:
        return "Senior"
age = int(input("Enter age: "))
print(f"Age group: {classify_age(age)}")
```

Output



Analysis of generated conditions and logic

- The nested if-elif-else structure effectively classifies ages into distinct groups.
- Each condition checks for specific age ranges, ensuring that every possible age input is categorized correctly.
- The conditions are clear and logically ordered from the youngest to the oldest age groups.

Explanation

The AI used if-elif-else logic to separate age ranges into meaningful categories. This demonstrates AI's capability in decision-making logic.

Observation

- Conditions were logically correct.
- AI did not address invalid age values.
- Shows AI prioritizes solving the main logic rather than edge cases.

Task 4: Sum of First n Numbers

Scenario

You need to calculate the sum of the first n natural numbers.

Task

• Use AI assistance to generate a sum_to_n() function using a for loop.

• Analyze the generated code.

• Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4

• Python function to compute the sum of first n numbers.

• Correct output for sample inputs.

• Explanation and comparison of different approaches.

Task Understanding

This task involved implementing multiple approaches to calculate the sum of natural numbers.

Steps Performed

- AI generated loop-based solutions.
- AI suggested a mathematical formula approach.

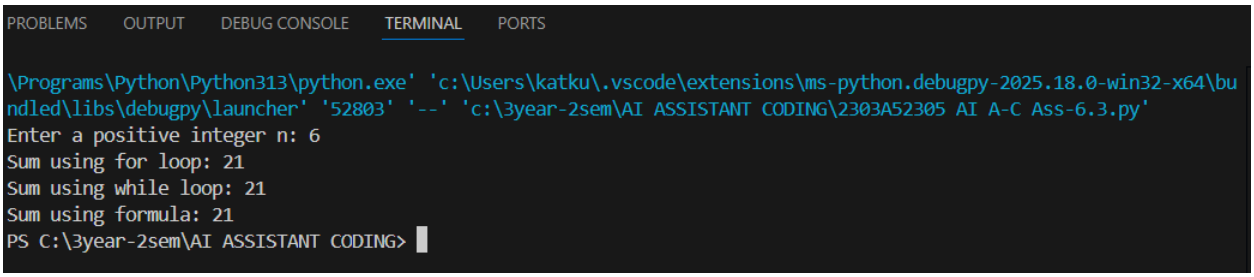
Prompt

Generate a code for Sum of First n Numbers

Code

```
def sum_to_n_for_loop(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
def sum_to_n_while_loop(n):
    total = 0
    i = 1
    while i <= n:
        total += i
        i += 1
    return total
def sum_to_n_formula(n):
    return n * (n + 1) // 2
n = int(input("Enter a positive integer n: "))
print(f"Sum using for loop: {sum_to_n_for_loop(n)}")
print(f"Sum using while loop: {sum_to_n_while_loop(n)}")
print(f"Sum using formula: {sum_to_n_formula(n)}")
```

Output



Explanation

Loop methods use iteration, while the formula approach directly calculates the sum efficiently.

Explanation and comparison of different approaches

- The for loop and while loop implementations both iterate through numbers from 1 to n,
- accumulating the sum. The for loop is more concise, while the while loop provides more control.
- The mathematical formula approach is the most efficient, as it computes the sum in constant timewithout any loops, making it ideal for large values of n.

Observation

- AI provided both procedural and optimized logic.
- Formula method was more efficient.
- AI only provides optimization when prompted.

Task 5: Bank Account Class

Scenario

You are designing a basic banking application.

Task

• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(),

and check_balance().

• Analyze the AI-generated class structure and logic.

• Add meaningful comments and explain the working of the code.

Expected Output #5

• Complete Python Bank Account class.

• Demonstration of deposit and withdrawal operations with updated balance.

• Well-commented code with a clear explanation.

Task Understanding

The goal was to design a class simulating banking operations.

Steps Performed

- AI generated class with deposit, withdrawal, and balance check methods.
- Tested with sample operations.

Prompt

Generate BankAccount class with deposit, withdraw, check_balance methods.

Code

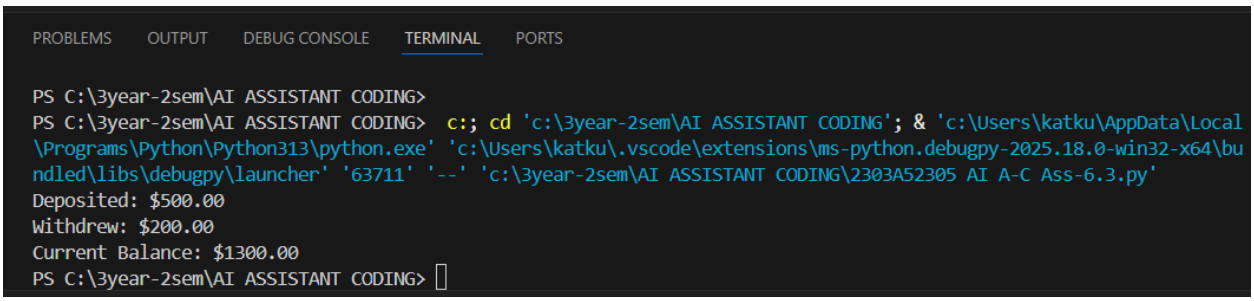
```
class BankAccount:
  def __init__(self, account_holder, initial_balance=0):
    self.account_holder = account_holder
    self.balance = initial_balance


  def deposit(self, amount):
    """Deposit the specified amount into the account."""
    if amount > 0:
      self.balance += amount
      print(f"Deposited: ${amount:.2f}")
    else:
      print("Deposit amount must be positive.")
```

```
    def withdraw(self, amount):
        """Withdraw the specified amount from the account if sufficient funds are available."""
        if amount > 0:
            if amount <= self.balance:
                self.balance -= amount
                print(f"Withdrew: ${amount:.2f}")
            else:
                print("Insufficient funds for this withdrawal.")
        else:
            print("Withdrawal amount must be positive.")

    def check_balance(self):
        """Return the current balance of the account."""
        return self.balance
# Sample usage
account = BankAccount("John Doe", 1000)
account.check_balance()
account.deposit(500)
account.withdraw(200)
print(f"Current Balance: ${account.check_balance():.2f}")
```

Output



Analysis of AI-generated class structure and logic

- The BankAccount class is well-structured, with an __init__ method to initialize the account holder's name and initial balance.
- The deposit() and withdraw() methods include checks for valid amounts and sufficient funds,ensuring robust handling of transactions.
- The check_balance() method provides a simple way to retrieve the current balance.
- The code is well-commented, making it easy to understand the purpose of each method and its functionality.

Explanation

AI structured class methods logically. Deposit adds funds, withdrawal subtracts with condition checks, and balance method displays results.

Observation

- AI structured OOP logic well.
- Security checks like transaction limits or password protection were not included.
- Highlights AI's limitation in security awareness.

Overall Lab Observations

Aspect	AI Strength	AI Limitation
Classes	Good structure	No data validation
Loops	Correct logic	No efficiency discussion
Conditionals	Clear logic	Edge cases missing
Optimization	Provides formula	Needs prompt
OOP Systems	Logical methods	Security gaps